CC3069 – Computación Paralela y Distribuida
Sección 10
Ing. Kimberly Barrera
Informe: Proyecto 3
Integrantes: Cindy Gualim 21226, María José Girón 23559, Leonardo Mejía 23648

Mejorando algoritmos con CUDA: reconocimiento del alfabeto ASL

Repositorio: https://github.com/DufreyM/CC3069-Proyecto3 (rama `main`). El código, los tiempos medidos, el monitoreo y los logs están en el repositorio; las cifras de este informe se generan a partir de `resultados/` con `python informe/build_informe.py`.

1. Introducción

Este proyecto toma el Laboratorio 3 de CC3084 Data Science, un clasificador del alfabeto de señas americano (ASL) con redes convolucionales, y lo reorganiza para aprovechar el hardware: la preparación de imágenes se reparte entre varios núcleos de la CPU y toda la matemática de la red neuronal se ejecuta en la GPU con CUDA. El objetivo es medir cuánto se reduce el tiempo de cada fase frente a la versión secuencial, con los mismos datos, modelos e hiperparámetros, y explicar por qué.

Se usa el dataset ASL Alphabet de Kaggle: 87,000 imágenes JPG de 200×200 en 29 clases (A-Z, `del`, `space` y `nothing`). Igual que en el laboratorio, se toman 650 imágenes por clase (18,850), divididas 70/15/15 en 13,195 de entrenamiento, 2,827 de validación y 2,828 de prueba, y se redimensionan a 64×64. La preparación también se midió con las 87,000 imágenes para observar el efecto del tamaño del problema.

2. Entorno y metodología

Componente	Detalle
CPU	Intel Core i9-13900HX: 24 núcleos (8 de rendimiento y 16 eficientes), 32 hilos lógicos
GPU	NVIDIA GeForce RTX 4060 Laptop, 8 GB de VRAM, controlador 581.42
Memoria y disco	32 GB de RAM; SSD NVMe SK Hynix de 1 TB
Software	Windows 11 Home, Python 3.12.14, PyTorch 2.11.0 con CUDA 12.8 y cuDNN, Numba 0.68.1 con numba-cuda 0.30.4, OpenCV 5.0, NumPy 2.3

El código separa estrictamente las dos fases. `src/preparacion_secuencial.py` y `src/preparacion_paralela.py` solo leen y transforman archivos en CPU y no importan PyTorch; `src/entrenamiento.py` recibe tensores ya preparados y nunca toca un archivo de imagen. Las mediciones se repitieron bajo las mismas condiciones (equipo conectado a la corriente, mismas semillas):

- **Preparación:** 5 corridas por cantidad de procesos con la muestra y 3 con el dataset completo. Antes se leen todos los archivos una vez para que ninguna corrida pague la lectura en frío. Cada resultado paralelo se compara con el secuencial mediante una huella SHA-256 del tensor.
- **Entrenamiento:** 15 épocas, lotes de 64, Adam con tasa 0.001. GPU: 5 corridas por modelo y modo. CPU con 24 hilos: 3 corridas. CPU con 1 hilo: 3 corridas de 3 épocas, porque 15 épocas tardarían cerca de media hora por corrida; su total de 15 épocas se estima sumando épocas estables y se marca con *.
- **Kernel CUDA:** 10 corridas sobre 1,024 imágenes reales; las cuatro versiones se validan contra PyTorch (error máximo < 10^{-4}).
- **Relojes:** `time.perf_counter`, con `torch.cuda.synchronize()` antes de detenerlo, porque la GPU trabaja de forma asíncrona. Los tiempos de GPU incluyen las transferencias CPU↔GPU. La primera época se registra aparte porque incluye la inicialización de CUDA.

La preparación de la muestra se midió mientras había otras aplicaciones abiertas (carga de CPU cercana al 20 % antes de empezar); se cerraron antes de medir el dataset completo y el resto de fases. Unas mediciones previas en modo de ahorro de energía dieron tiempos de hasta el doble y la GPU limitada a 40 W, por lo que se descartaron.

3. Análisis de la solución secuencial y diseño paralelo

Pasos 1, 3 y 4 del enunciado. La solución base es el Laboratorio 3 de CC3084 Data Science (reconocimiento del alfabeto ASL con CNN), formado por dos notebooks:

Parte	Función
`preprocesamiento.ipynb`	Lista las 87,000 imágenes, toma 650 por clase, asigna etiquetas, divide 70/15/15, carga y normaliza cada imagen y guarda los tensores en `.npy`.
`modelado.ipynb`	Carga los tensores, los envuelve en `Dataset`/`DataLoader`, define las CNN, las entrena 15 épocas, evalúa en test y compara con otros modelos.

Este proyecto conserva lo que mide rendimiento (preparación y entrenamiento de las dos CNN) y deja fuera la red fully-connected, el Random Forest, el data augmentation y las fotos propias.

3.1 Operaciones del programa

No.	Operación identificada	Descripción de su funcionamiento
1	Inventario de archivos	Recorre las 29 carpetas de clase y lista los 87,000 JPG.
2	Muestreo estratificado	Toma 650 imágenes por clase con semilla 42 (18,850 en total).
3	Asignación de etiquetas	Convierte cada clase en un entero de 0 a 28.
4	División train/val/test	Separa 13,195 / 2,827 / 2,828 imágenes de forma estratificada.
5	Lectura y decodificación	`cv2.imread` lee el archivo del disco y descomprime el JPG a una matriz 200×200×3.
6	Conversión de color	Reordena los canales de BGR a RGB.
7	Redimensionado	Interpola de 200×200 a 64×64 píxeles.
8	Normalización	Convierte a `float32` y divide entre 255 para dejar valores en [0, 1].
9	Construcción del tensor	Apila las imágenes en un arreglo (N, 64, 64, 3).
10	Persistencia	Guarda los tensores en `.npy` para no repetir la preparación.
11	Formato para PyTorch	Transpone a (N, 3, 64, 64) y arma lotes de 64 con `DataLoader`.
12	Propagación hacia adelante	Convoluciones, ReLU, BatchNorm, MaxPool, Dropout y capas densas producen 29 puntajes por imagen.
13	Función de pérdida	Entropía cruzada entre los puntajes y la etiqueta real.
14	Retropropagación	Calcula el gradiente de la pérdida respecto a cada peso.
15	Actualización de pesos	Adam ajusta todos los parámetros con tasa 0.001.
16	Validación	Al final de cada época, propaga el set de validación sin calcular gradientes.
17	Evaluación	Predice el set de prueba y calcula accuracy y F1 macro.

3.2 Operaciones repetitivas

Valores para la muestra de 18,850 imágenes y 15 épocas. Los MAC (multiplicación + suma) por imagen se contaron sobre las capas convolucionales y densas de cada modelo.

Operación repetitiva	Datos que procesa	Cantidad de repeticiones
Lectura y decodificación JPG (5)	Un archivo de unos 13 KB → 200×200×3 = 120,000 valores	18,850 (87,000 con el dataset completo)
Conversión de color (6)	40,000 píxeles por imagen	18,850
Redimensionado (7)	120,000 valores de entrada → 12,288 de salida	18,850
Normalización (8)	12,288 valores por imagen	18,850 imágenes = 231.6 millones de divisiones
Paso de entrenamiento (12-15)	Lote de 64 imágenes de 3×64×64	207 lotes × 15 épocas = 3,105
Convolución, CNN básica (12)	18.8 millones de MAC por imagen (92.1 % del total de 20.4 M)	13,195 imágenes × 15 épocas = 197,925 propagaciones, más su retropropagación
Convolución, CNN profunda (12)	116.8 millones de MAC por imagen (98.2 % del total de 118.9 M)	197,925 propagaciones, más su retropropagación
Actualización Adam (15)	1,628,893 parámetros (básica) o 2,244,477 (profunda)	3,105 por modelo
Validación (16)	2,827 imágenes en 45 lotes	15 por modelo

Solo en propagación hacia adelante, la CNN profunda hace unos 1.57 billones de MAC por época (13,195 × 118.9 M); la retropropagación cuesta aproximadamente el doble.

3.3 Oportunidades de paralelización

Operación	¿Paralelizable?	Dónde	Justificación
Inventario, muestreo y división (1-4)	No conviene	CPU, 1 proceso	Se ejecutan una vez y tardan poco frente al resto.
Lectura, decodificación, color, redimensionado y normalización (5-8)	Sí	CPU, varios procesos	Cada imagen es independiente (paralelismo de datos). La lectura toca el disco y la decodificación JPG es lógica secuencial con muchas ramas; ninguna de las dos aprovecha bien miles de hilos de GPU.
Construcción del tensor (9)	Sí, sin costo extra	CPU, memoria compartida	Cada proceso escribe su bloque directo en el tensor final, así no hay que copiar ni serializar resultados.
Propagación, pérdida, retropropagación y Adam (12-15)	Sí	GPU con CUDA	Son operaciones de álgebra lineal sobre miles de valores independientes: cada píxel de salida de una convolución se calcula con la misma fórmula sobre datos distintos (SIMT). Es el 92-98 % del cómputo.
Validación y evaluación (16-17)	Sí	GPU con CUDA	Es la misma propagación hacia adelante, sin gradientes.
Ciclo de épocas	No	Secuencial	Cada época necesita los pesos de la anterior; solo se paraleliza dentro de cada paso.

3.4 Diseño del algoritmo paralelo

Fase de CPU: preparación de imágenes

1. El proceso principal crea un tensor vacío de (N, 64, 64, 3) en memoria compartida.
2. La lista de rutas se divide en bloques de 256 imágenes contiguas.
3. Un grupo de P procesos (`ProcessPoolExecutor`) toma bloques a medida que se desocupa, lo que reparte la carga aunque el i9-13900HX mezcle núcleos rápidos (P) y eficientes (E).
4. Cada proceso aplica las operaciones 5-8 y escribe cada imagen en su posición; el orden final es idéntico al secuencial, y se comprueba con una huella SHA-256 del tensor.
5. Se usan procesos y no hilos para que el GIL de Python no serialice el trabajo. OpenCV y OpenBLAS se limitan a 1 hilo por proceso para no sobresuscribir los núcleos.

Fase de GPU: entrenamiento

Los tensores ya preparados se envían al GPU por lotes de 64 imágenes (`pin_memory` + copia asíncrona) o completos a la VRAM en el modo optimizado. PyTorch delega cada capa a cuDNN, que elige la configuración de hilos y bloques. Para mostrar ese reparto explícitamente se implementó la

primera convolución de la CNN profunda como kernel propio en CUDA Python (Numba, `src/conv_cuda.py`):

Nivel	Asignación
Hilo	Calcula un píxel de salida (i, j) de un filtro k para una imagen n: 27 MAC (3 canales × 3×3) más el sesgo y ReLU.
Bloque	16×16 = 256 hilos que cubren un parche de 16×16 píxeles de la misma imagen y el mismo filtro. Los 27 pesos del filtro se cargan una sola vez en memoria compartida.
Grid	(⌈64/16⌉, ⌈64/16⌉, N·K) = (4, 4, N·32). Los ejes x, y recorren la imagen y el eje z cada pareja (imagen, filtro). Un lote de 64 imágenes lanza 4×4×2,048 = 32,768 bloques, es decir 8.4 millones de hilos.

Como cada hilo escribe una salida distinta, no hay condiciones de carrera ni se necesitan operaciones atómicas; la única barrera (`syncthreads`) asegura que los pesos estén en memoria compartida antes de usarlos.

4. Resultados

4.1 Resumen y speedup por fase

Fase	Secuencial (s)	Paralelo (s)	Paralelo con	Speedup
Preparación de imágenes (18,850)	5.1	1.6	CPU, 8 procesos	3.2×
Entrenamiento CNN profunda (15 épocas)	1,701.5	44.4	GPU con CUDA	38.3×
Pipeline completo	1,706.6	46.0	CPU paralela + GPU	37.1×

El pipeline completo (preparar 18,850 imágenes y entrenar la CNN profunda 15 épocas) pasa de unos 28 minutos en un solo núcleo a 46 segundos combinando procesos de CPU para la preparación y CUDA para el entrenamiento. Como el entrenamiento ocupa más del 99 % del tiempo secuencial, el speedup total lo determina casi por completo la GPU.

4.2 Preparación de imágenes en CPU

Conjunto	Procesos	Corridas	Promedio (s)	Desv. (s)	Speedup	Eficiencia
Completo (87,000)	1	3	27.55	1.42	1.00×	100%
Completo (87,000)	2	3	12.88	0.10	2.14×	107%
Completo (87,000)	4	3	8.45	0.04	3.26×	82%
Completo (87,000)	8	3	6.29	0.04	4.38×	55%
Completo (87,000)	12	3	6.05	0.23	4.55×	38%
Completo (87,000)	16	3	6.25	0.03	4.40×	28%
Completo (87,000)	24	3	7.27	0.04	3.79×	16%
Completo (87,000)	32	3	7.68	0.03	3.59×	11%
Muestra (18,850)	1	5	5.11	0.57	1.00×	100%
Muestra (18,850)	2	5	3.15	0.15	1.62×	81%
Muestra (18,850)	4	5	2.05	0.02	2.50×	62%
Muestra (18,850)	8	5	1.60	0.02	3.19×	40%
Muestra (18,850)	12	5	1.61	0.03	3.17×	26%
Muestra (18,850)	16	5	1.83	0.09	2.80×	17%
Muestra (18,850)	24	5	2.23	0.04	2.29×	10%
Muestra (18,850)	32	5	2.51	0.03	2.04×	6%

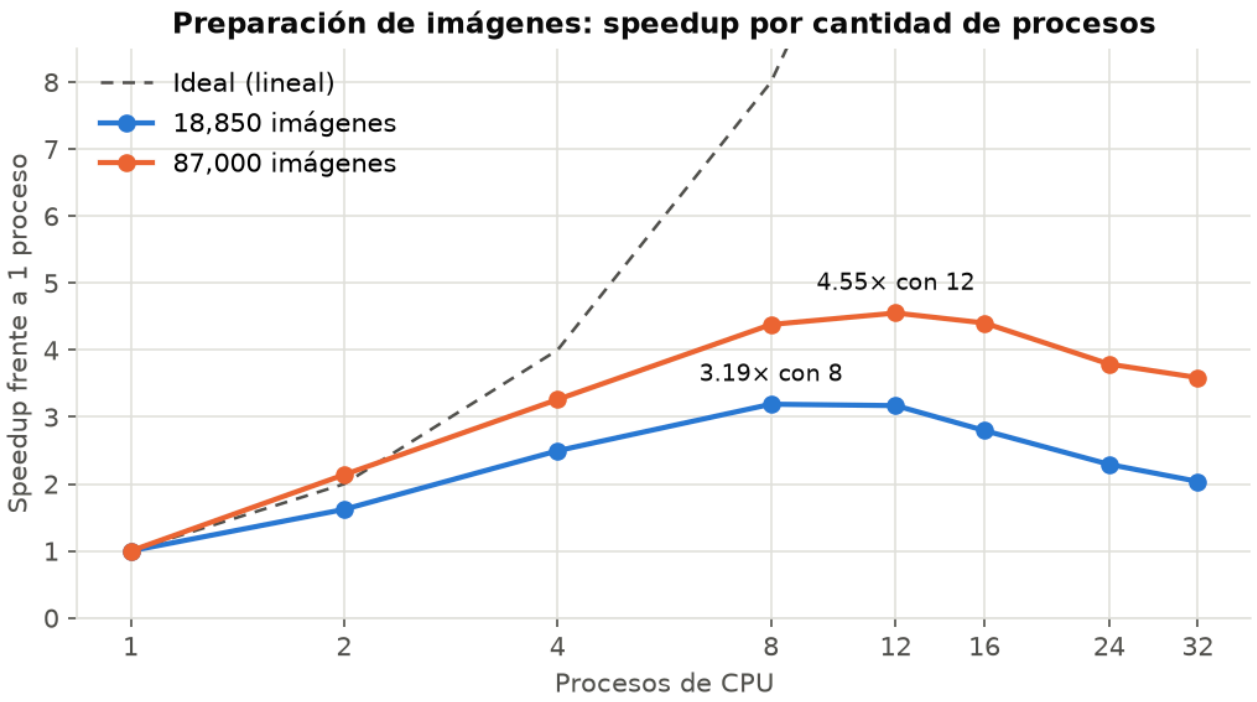


Figura 1. Speedup de la preparación según la cantidad de procesos.

La versión secuencial procesa unas 3,700 imágenes por segundo en un núcleo. Con la muestra, el mejor tiempo se logra con 8 procesos (3.19×, eficiencia del 40 %); con el dataset completo, con 12 procesos (4.55×). A partir de 12-16 procesos el tiempo vuelve a subir. Esto se debe a que cada proceso nuevo en Windows arranca un intérprete e importa OpenCV y NumPy (un costo fijo que pesa más cuando el trabajo total dura solo 1.6 s), a que todos los procesos comparten el mismo disco y el mismo bus de memoria, y a que pasados los 8 núcleos de rendimiento el trabajo cae en hilos de hyperthreading y núcleos eficientes, más lentos. Con más imágenes el costo fijo se reparte mejor, por eso el dataset completo escala más.

4.3 Entrenamiento: CPU frente a GPU

Modelo	Modo	Corridas	s / época	1.ª época (s)	Total 15 ép. (s)	Speedup vs 1 hilo	Speedup vs 24 hilos	Accuracy test
CNN básica	CPU 1 hilo	3 × 3 ép.	34.22	34.54	517.7 *	1.0×	0.3×	0.7432
CNN básica	CPU 24 hilos	3 × 15 ép.	9.10	7.67	136.3	3.8×	1.0×	0.9182
CNN básica	GPU (CUDA)	5 × 15 ép.	1.06	1.15	16.7	31.0×	8.2×	0.9172
CNN básica	GPU optimizada	5 × 15 ép.	0.72	1.02	11.5	45.1×	11.9×	0.9202
CNN profunda	CPU 1 hilo	3 × 3 ép.	112.86	113.17	1,701.5 *	1.0×	0.3×	0.8256
CNN profunda	CPU 24 hilos	3 × 15 ép.	37.11	35.45	556.7	3.1×	1.0×	0.9771
CNN profunda	GPU (CUDA)	5 × 15 ép.	2.92	3.04	44.4	38.3×	12.5×	0.9748
CNN profunda	GPU optimizada	5 × 15 ép.	1.87	2.12	28.9	58.8×	19.2×	0.9801

* Total estimado: corridas de 3 épocas llevadas a 15 sumando épocas estables. El accuracy de esas corridas es bajo porque solo entrenaron 3 épocas; no refleja una diferencia de calidad.

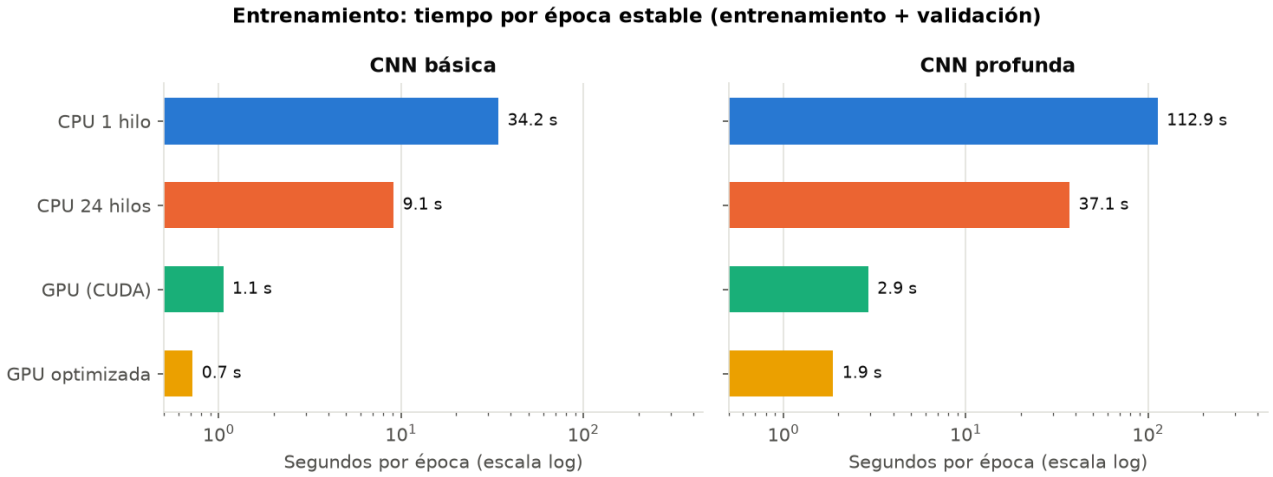


Figura 2. Tiempo por época estable en cada modo (escala logarítmica).

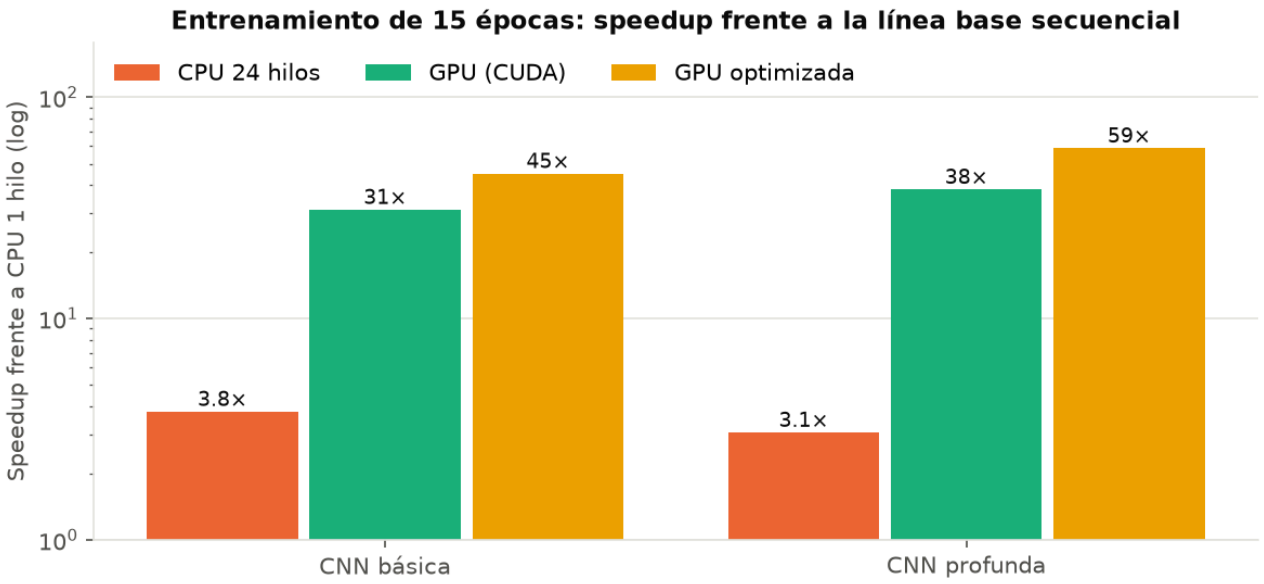


Figura 3. Speedup del entrenamiento de 15 épocas frente a la CPU con 1 hilo.

Para la CNN profunda, una época pasa de 112.9 s en un hilo a 37.1 s con 24 hilos de CPU (3.0×), 2.92 s en GPU (38.7×) y 1.88 s en el modo optimizado (60.2×). Frente a la mejor configuración de CPU, la GPU sigue siendo 12.5× más rápida y la versión optimizada 19.2×. La CNN básica gana menos (31× y 45×) porque hace 5.8 veces menos operaciones por imagen y la GPU queda más tiempo esperando datos. El accuracy en test con 15 épocas es equivalente entre CPU y GPU (0.977, 0.975 y 0.980 en la profunda), así que la aceleración no cambia el resultado del modelo.

4.4 ¿En qué se va el tiempo en la GPU?

Parte del paso	Segundos	%
Obtener lote (CPU)	1.51	9.5
Copia CPU→GPU	0.36	2.3
Cómputo en GPU	13.98	88.2

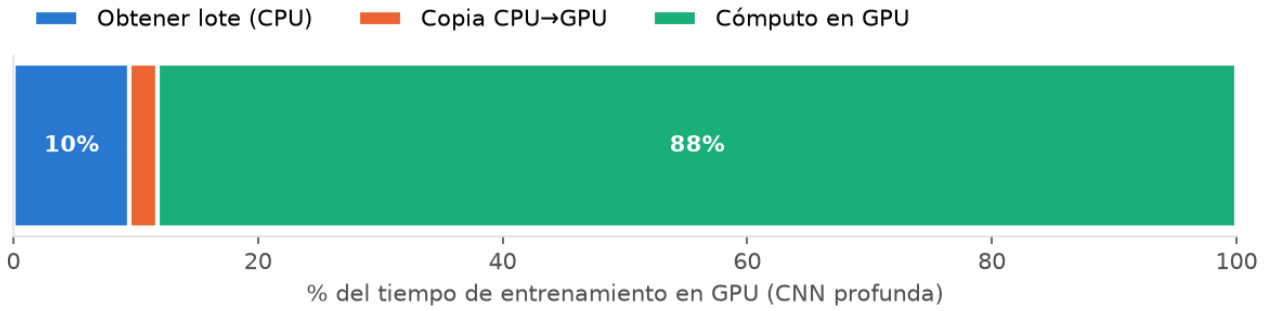


Figura 4. Reparto del tiempo de un paso de entrenamiento en GPU.

Con el `DataLoader` original, el 88 % del paso es cómputo en GPU, el 9.5 % es la CPU armando el lote y solo el 2.3 % es la copia CPU→GPU. El modo optimizado elimina el armado de lotes al cargar los tensores completos en VRAM una sola vez (1 GB) y usa precisión mixta (AMP), que aprovecha los Tensor Cores: así baja de 2.92 a 1.88 s por época.

4.5 Kernel CUDA propio: la primera convolución

Versión	Total con copias (ms)	Solo kernel (ms)	Speedup total	Speedup solo kernel
CPU 1 hilo (Numba)	1312.7	—	1.0×	—
CPU paralela (Numba prange)	144.5	—	9.1×	—
Kernel CUDA propio (Numba)	133.9	18.2	9.8×	72.2×
cuDNN (PyTorch)	82.8	—	15.8×	—

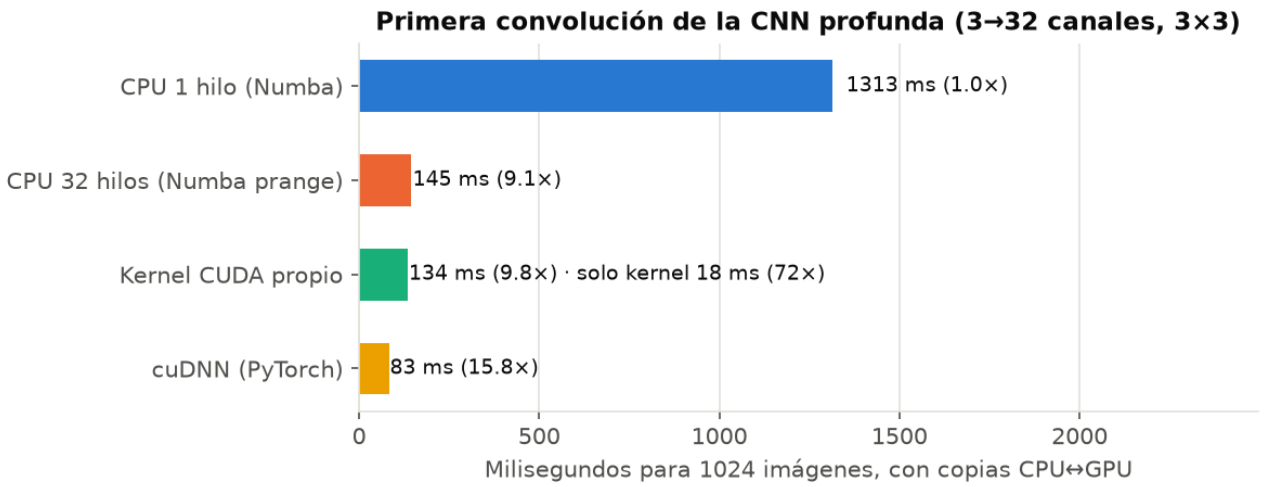


Figura 5. Primera convolución: CPU, kernel CUDA propio y cuDNN.

El kernel propio calcula la convolución 72 veces más rápido que el mismo bucle en un núcleo, pero contando las copias el speedup cae a 9.8×: de 134 ms, solo 18 ms son cómputo y el 86 % restante es copiar 50 MB de entrada al GPU y traer de vuelta 537 MB de salida. Con transferencias, el kernel propio apenas supera a la CPU con 32 hilos (144 ms), y cuDNN es más rápido (83 ms) porque usa algoritmos de convolución más eficientes que el bucle directo. Esto muestra por qué en el entrenamiento real los datos deben quedarse en la GPU entre capas: si cada capa volviera a la CPU, las transferencias anularían la ganancia.

5. Evidencia de ejecución

Un hilo de monitoreo registró cada 0.5 s el uso de los 32 núcleos lógicos, la RAM, el uso de GPU, la VRAM y la potencia (`psutil` y NVML) durante todas las mediciones.

Medición	CPU prom. (%)	Núcleos > 50 %	GPU prom. (%)	VRAM máx. (GB)	Potencia GPU máx. (W)
entrenamiento_cpu_1hilo_profunda	5.4	1	0.0	0.23	4.6
entrenamiento_cpu_nhilos_profunda	77.4	31	0.0	0.23	5.3
entrenamiento_cuda_optimizado_profunda	13.3	2	92.8	1.49	76.8
entrenamiento_cuda_profunda	79.5	32	97.4	0.71	81.8
preparacion_completo_p1	12.5	2	0.0	0.23	3.8
preparacion_completo_p12	28.3	6	0.0	0.23	2.8

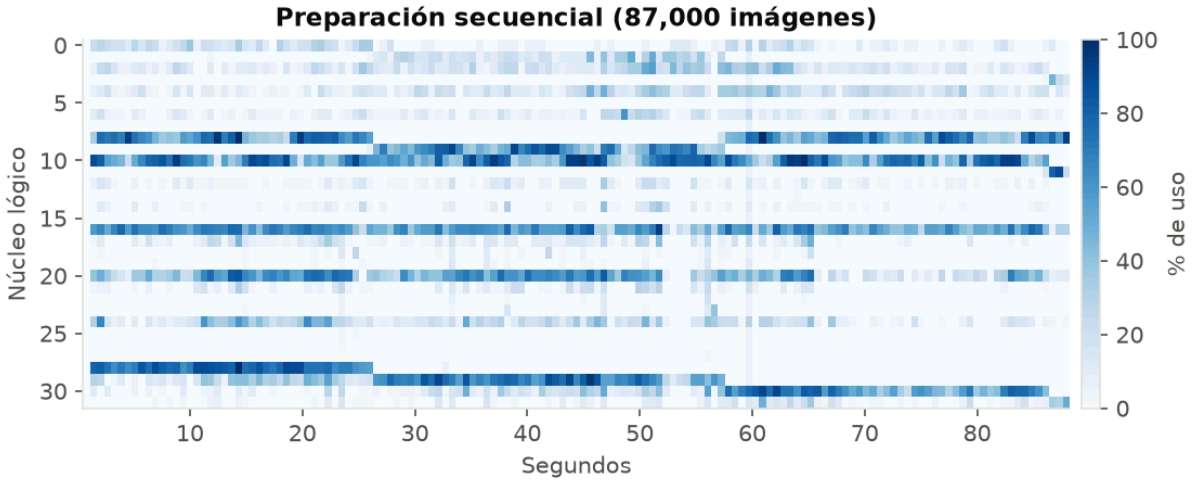


Figura 6. Uso por núcleo durante la preparación secuencial (87,000 imágenes).

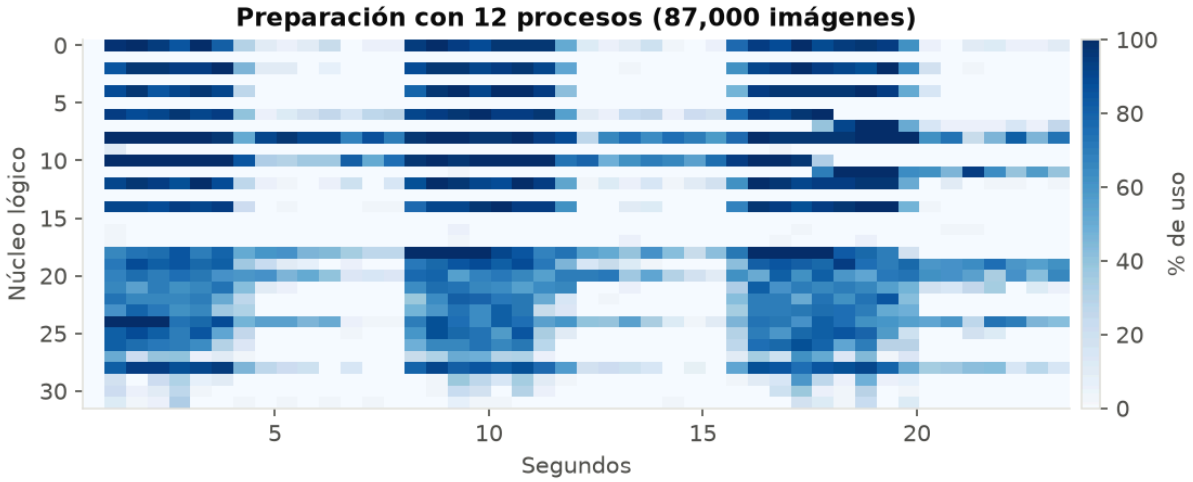


Figura 7. Uso por núcleo durante la preparación con 12 procesos (87,000 imágenes).

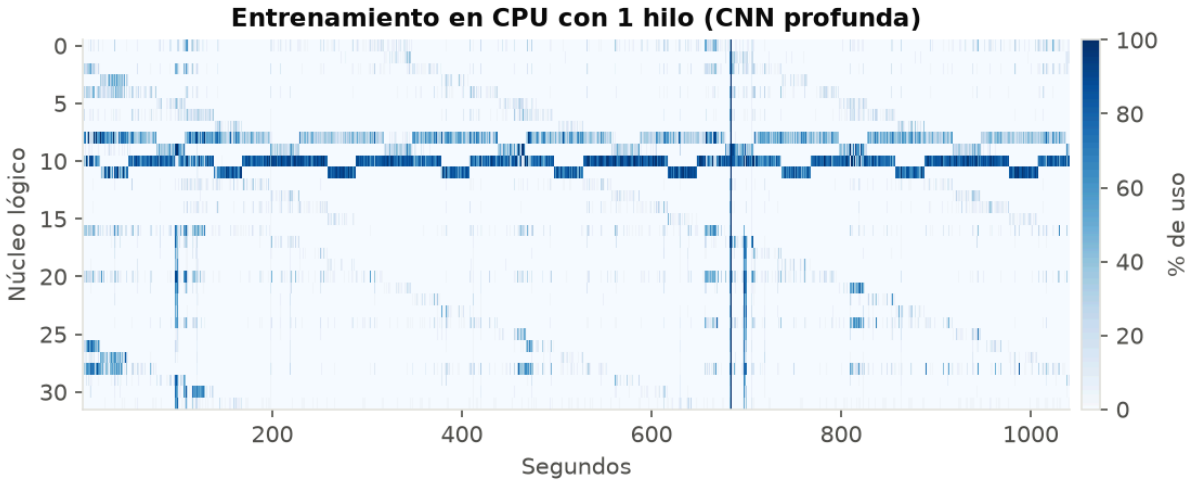


Figura 8. Uso por núcleo al entrenar en CPU con 1 hilo.

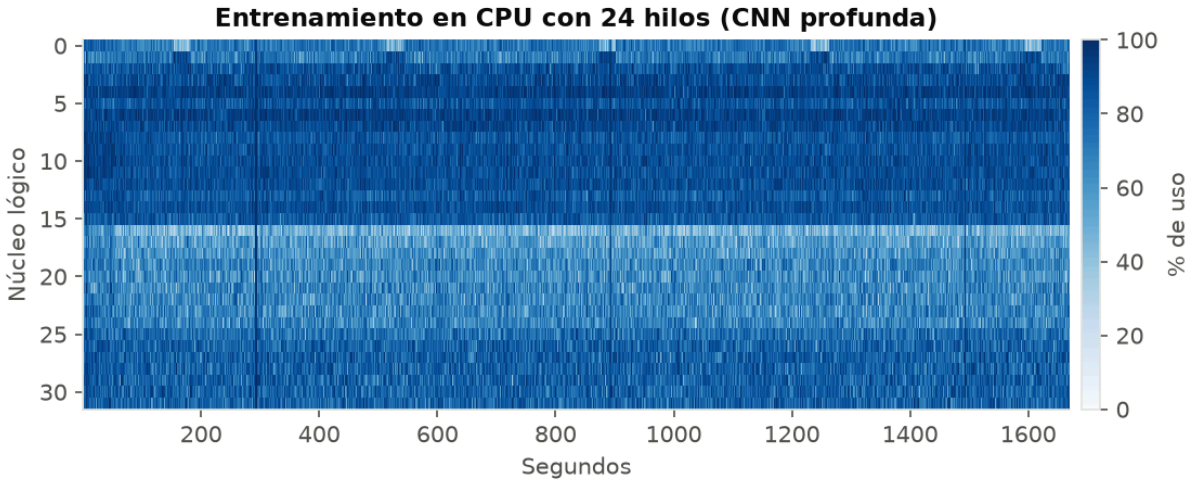


Figura 9. Uso por núcleo al entrenar en CPU con 24 hilos.

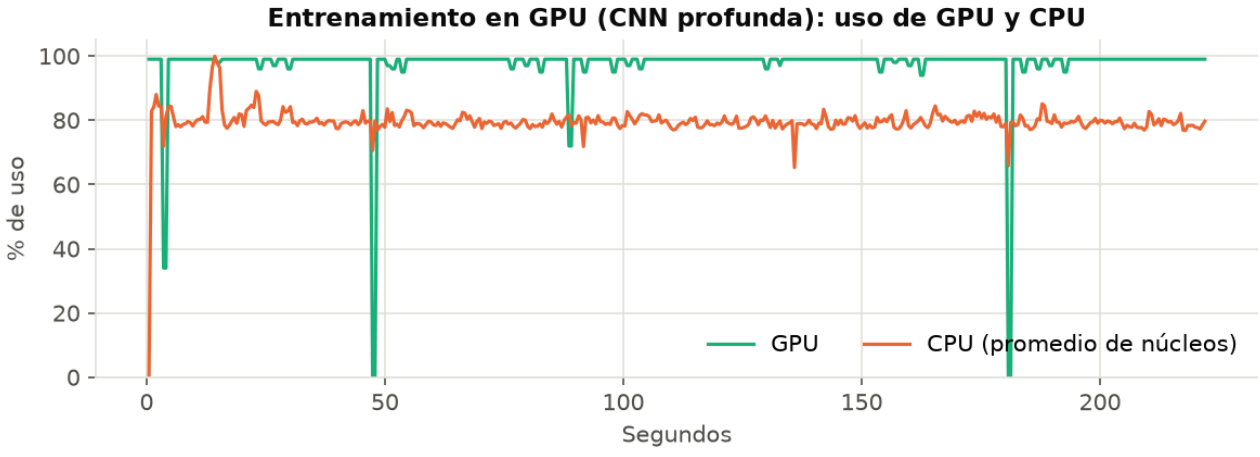


Figura 10. Uso de GPU y CPU al entrenar con CUDA.

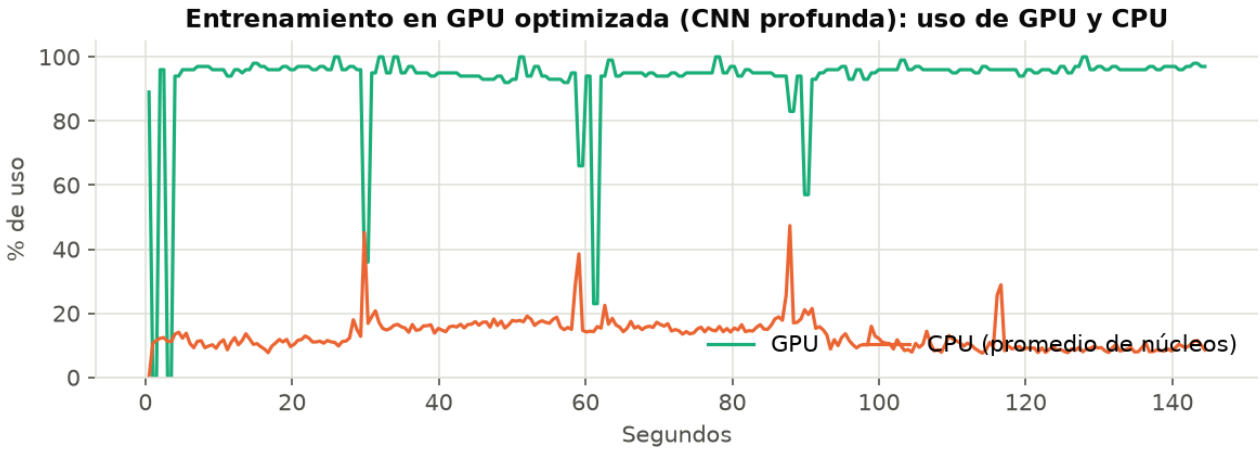
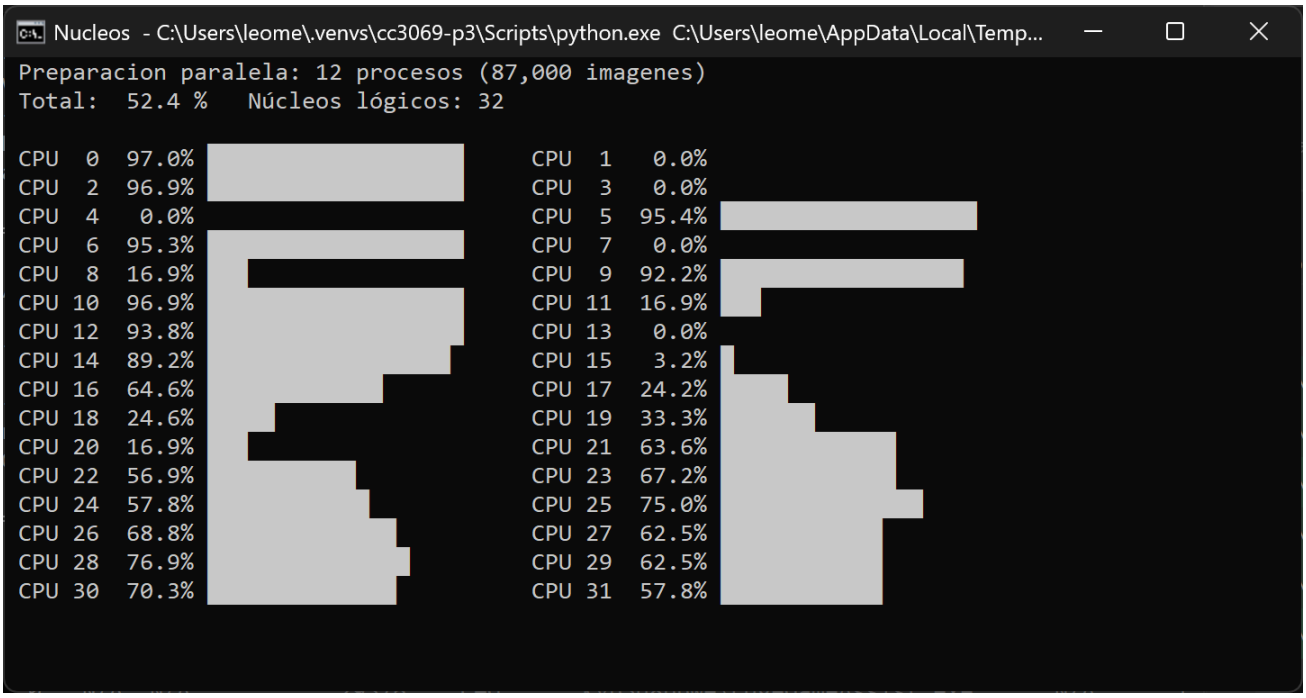


Figura 11. Uso de GPU y CPU en el modo GPU optimizado.

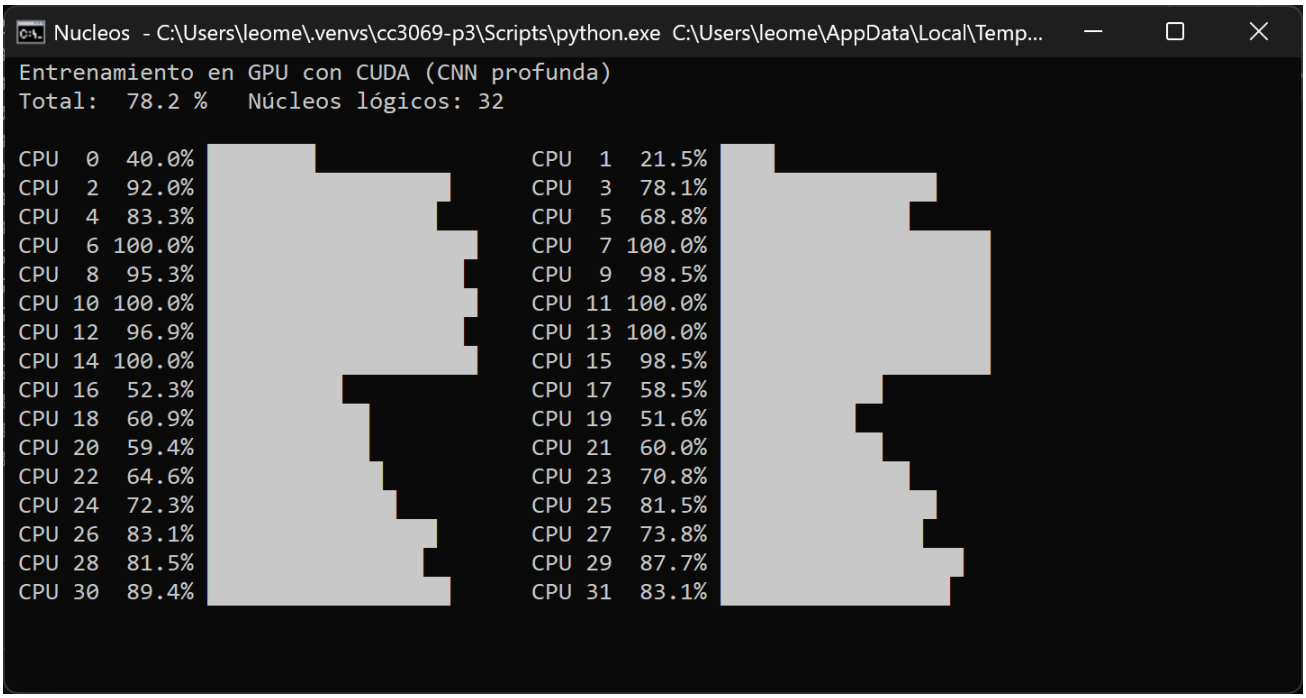
Las figuras confirman que cada fase usó el hardware esperado. Con un hilo, solo un núcleo supera el 50 % de uso (el sistema lo va moviendo entre núcleos); con 24 hilos, 31 de los 32 núcleos lógicos superan el 50 % y la GPU queda en 0 %. En GPU, la utilización promedio es del 97 % con 0.7 GB de VRAM y hasta 82 W, en estado P0. Un dato llamativo: en el modo `cuda` la CPU sigue al 80 %, porque el `DataLoader` arma cada lote imagen por imagen usando los 24 hilos de PyTorch; en el modo optimizado la CPU baja al 13 % y la GPU mantiene un 93 %.

Capturas de pantalla

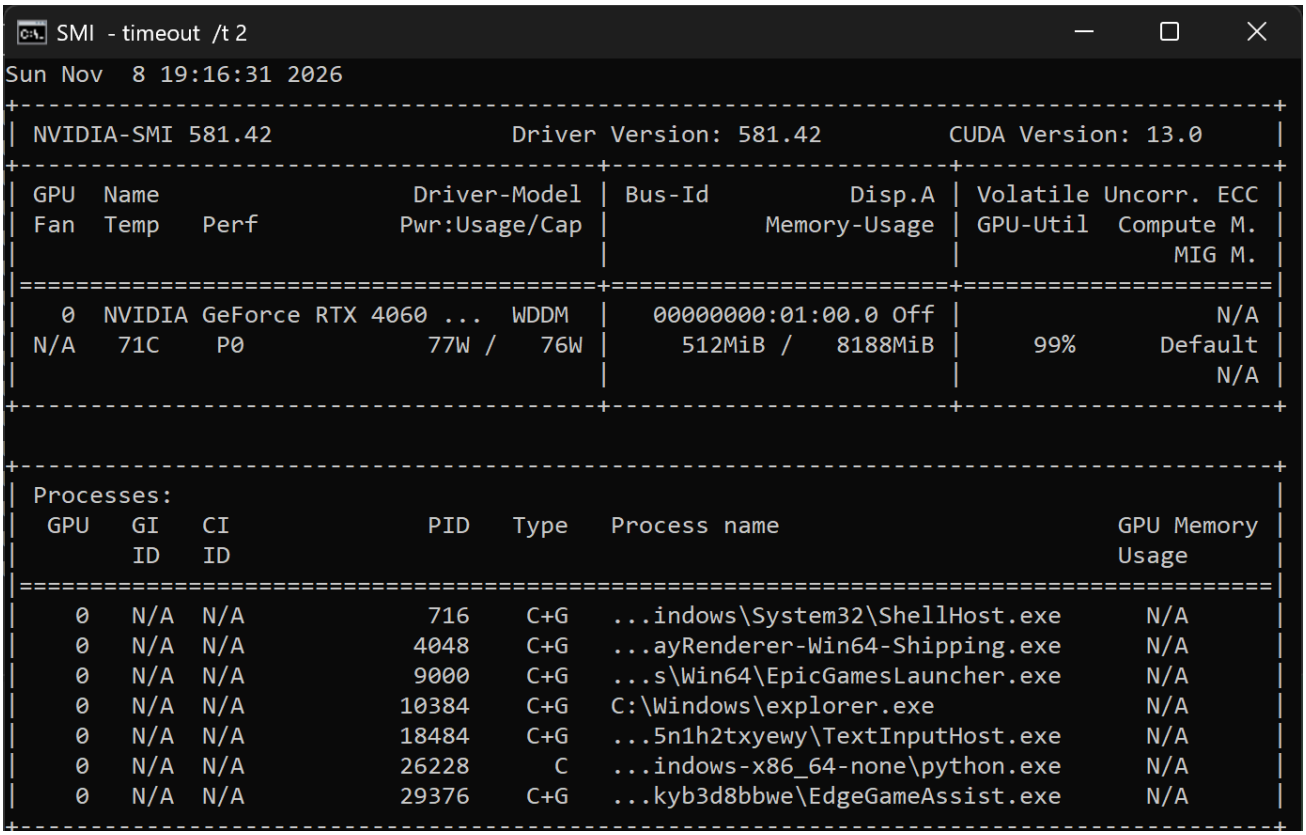
Capturas tomadas mientras corría cada fase. El uso por núcleo se muestra con un monitor de consola que lee los contadores del sistema operativo cada segundo (`psutil`), y el estado de la GPU con `nvidia-smi`. Durante la preparación paralela la carga se reparte entre muchos núcleos; durante el entrenamiento en GPU, `nvidia-smi` muestra el proceso `python.exe` con 99 % de uso, 77 W y estado P0, mientras la CPU arma los lotes; con un hilo de CPU, solo unos pocos núcleos tienen carga.



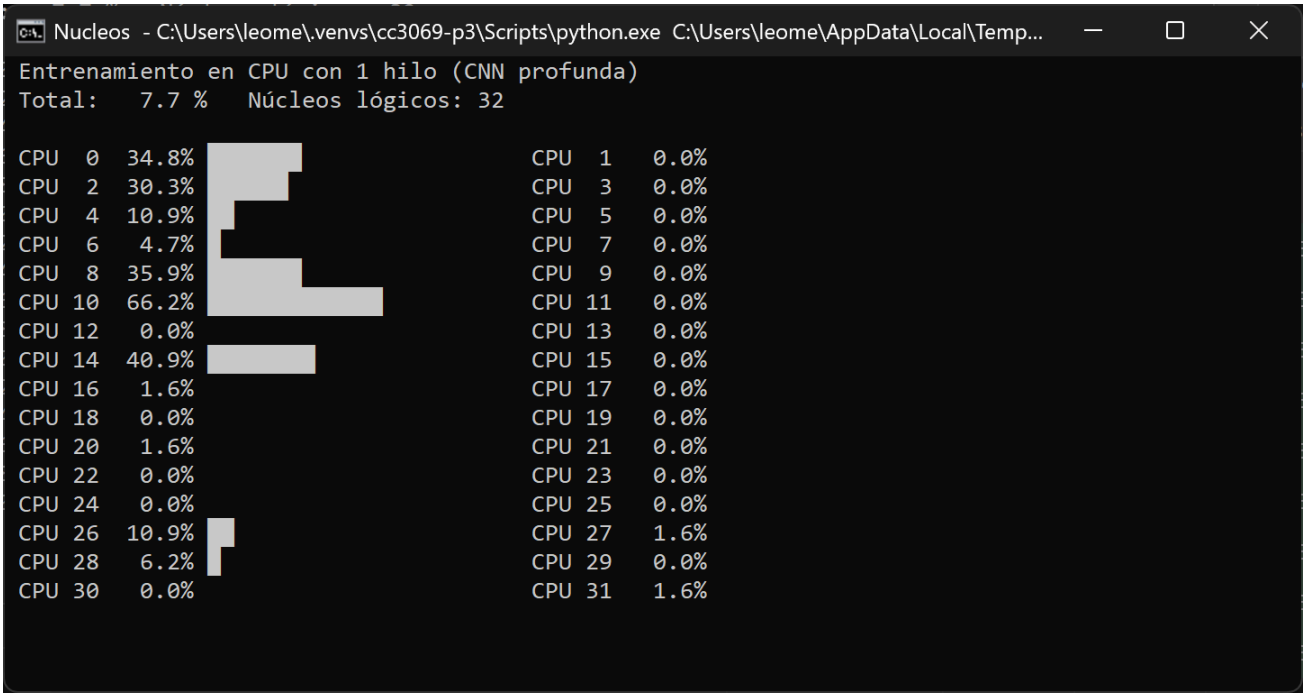
Captura 1 - Uso por nucleo durante la preparacion paralela (12 procesos)



Captura 2 - Uso por nucleo durante el entrenamiento en GPU



Captura 3 - nvidia-smi durante el entrenamiento en GPU (99 %, 77 W, python.exe)



Captura 4 - Uso por nucleo durante el entrenamiento en CPU con 1 hilo

Logs

nvidia-smi durante el entrenamiento en GPU (CNN profunda)

```
Sun Nov  8 16:16:57 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 581.42                 Driver Version: 581.42         CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4060 ...  WDDM  |   00000000:01:00.0 Off |                  N/A |
| N/A   85C    P0             74W /   74W |     493MiB /   8188MiB |     99%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A             716    C+G   ...indows\System32\ShellHost.exe      N/A      |
|    0   N/A  N/A            4048    C+G   ...ayRenderer-Win64-Shipping.exe      N/A      |
|    0   N/A  N/A            9000    C+G   ...s\Win64\EpicGamesLauncher.exe      N/A      |
|    0   N/A  N/A           10384    C+G   C:\Windows\explorer.exe               N/A      |
|    0   N/A  N/A           18484    C+G   ...5n1h2txyewy\TextInputHost.exe      N/A      |
|    0   N/A  N/A           29376    C+G   ...kyb3d8bbwe\EdgeGameAssist.exe      N/A      |
|    0   N/A  N/A           39476      C   ...indows-x86_64-none\python.exe      N/A      |
+-----------------------------------------------------------------------------------------+
```

nvidia-smi durante el entrenamiento en CPU (GPU sin carga)

```
Sun Nov  8 16:48:29 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 581.42                 Driver Version: 581.42         CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 4060 ...  WDDM  |   00000000:01:00.0 Off |                  N/A |
| N/A   55C    P8              1W /   64W |       0MiB /   8188MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A             716    C+G   ...indows\System32\ShellHost.exe      N/A      |
|    0   N/A  N/A            4048    C+G   ...ayRenderer-Win64-Shipping.exe      N/A      |
|    0   N/A  N/A            9000    C+G   ...s\Win64\EpicGamesLauncher.exe      N/A      |
|    0   N/A  N/A           10384    C+G   C:\Windows\explorer.exe               N/A      |
|    0   N/A  N/A           18484    C+G   ...5n1h2txyewy\TextInputHost.exe      N/A      |
|    0   N/A  N/A           29376    C+G   ...kyb3d8bbwe\EdgeGameAssist.exe      N/A      |
+-----------------------------------------------------------------------------------------+
```

Log del entrenamiento en GPU

```
PyTorch 2.11.0+cu128 | CUDA 12.8 | Python 3.12.14
GPU: NVIDIA GeForce RTX 4060 Laptop GPU
Hilos por defecto de PyTorch: 24
{'train': (13195, 64, 64, 3), 'val': (2827, 64, 64, 3), 'test': (2828, 64, 64, 3)}
cuda basica rep=1: 17.9 s | train 14.7 s | acc 0.9219 | f1 0.9223 | VRAM 0.14 GB
cuda basica rep=2: 16.1 s | train 14.4 s | acc 0.9148 | f1 0.9154 | VRAM 0.14 GB
cuda basica rep=3: 16.2 s | train 14.3 s | acc 0.9127 | f1 0.9124 | VRAM 0.14 GB
cuda basica rep=4: 16.3 s | train 14.6 s | acc 0.9077 | f1 0.9085 | VRAM 0.14 GB
```

Log del kernel CUDA

```
Entrada (1024, 3, 64, 64) | salida (1024, 32, 64, 64) | grid (4, 4, 32768) | bloque (16, 16) | hilos 134,217,728 | n
cpu_secuencial rep=1: 1.3935 s
cpu_secuencial rep=2: 1.3717 s
cpu_secuencial rep=3: 1.3485 s
```

6. Discusión técnica

¿Por qué procesar las imágenes con varios núcleos de CPU y delegar a la GPU solo la matemática de la red?

Porque cada fase tiene una forma de trabajo distinta. Preparar una imagen implica leer un archivo del sistema de archivos (que administra el sistema operativo en la CPU) y decodificar un JPG, cuyo formato comprimido se descifra de manera secuencial y con muchas ramas condicionales. Son tareas independientes entre imágenes pero irregulares por dentro, ideales para unos pocos núcleos potentes trabajando en paralelo: cada proceso toma su bloque de imágenes y no necesita coordinarse con los demás. Llevar estos archivos a la GPU no ahorraría nada: igual habría que leerlos del disco con la CPU y copiarlos por el bus PCIe, y la decodificación no aprovecharía miles de hilos.

La red neuronal es lo contrario: millones de multiplicaciones y sumas idénticas sobre datos distintos (116.8 millones solo en las convoluciones de una imagen de la CNN profunda). Ese paralelismo de datos regular es justo lo que la GPU resuelve con miles de hilos ejecutando la misma instrucción. Separar así las fases deja que cada procesador haga aquello para lo que fue diseñado.

¿Cómo CUDA permitió reducir el tiempo de procesamiento?

CUDA permite ejecutar cada capa como un kernel lanzado sobre millones de hilos: en el kernel propio, un lote de 64 imágenes lanza 32,768 bloques de 256 hilos, uno por cada píxel de salida de cada filtro. Mientras la CPU procesa unas pocas operaciones por ciclo en cada núcleo, la GPU las reparte entre sus 3,072 núcleos CUDA y oculta la latencia de memoria alternando entre grupos de hilos. PyTorch delega estas operaciones a cuDNN, que además elige algoritmos de convolución optimizados, y la precisión mixta usa los Tensor Cores. El resultado fue un entrenamiento de la CNN profunda 38× más rápido que en un hilo de CPU y 12.5× más rápido que con los 24 hilos (59× y 19× en el modo optimizado), sin cambiar el accuracy.

¿Qué factores justifican o limitan la utilización de CUDA?

La justifican: un volumen grande de cálculo regular (las convoluciones son el 92-98 % de las operaciones), datos que caben en VRAM (los tensores ocupan 0.9 GB de 8 GB) y un entrenamiento repetido muchas veces (3,105 pasos de optimización), donde el costo inicial se amortiza.

La limitan:

- **Transferencias:** en el kernel aislado, las copias fueron el 86 % del tiempo. CUDA solo rinde si los datos se quedan en la GPU entre operaciones.
- **Preparación del lado de la CPU:** con el `DataLoader`, la CPU arma los lotes al 80 % de uso y es el 9.5 % del paso; eliminarlo dio 1.56× adicional.
- **Modelos pequeños:** la CNN básica obtuvo menos speedup que la profunda porque no alcanza a ocupar toda la GPU.
- **Inicialización:** crear el contexto CUDA y cargar kernels toma segundos; con `cudnn.benchmark` activado la primera época tardaba de 5 a 11 s más, sin mejorar el tiempo por lote, por lo que se desactivó.
- **Energía y temperatura:** en una laptop el rendimiento depende del modo de energía. Con el equipo en ahorro, la GPU quedó limitada a 40 W y los tiempos casi se duplicaron; conectada llegó a 82 W y 85 °C.
- **Capacidad y entorno:** la VRAM de 8 GB limita el tamaño de lote y de dataset en memoria; además, combinar librerías fue delicado (PyTorch y Numba traen versiones distintas de la misma DLL de CUDA y hubo que controlar el orden de carga).

Funciones y librerías que permitieron trabajar con la GPU

Librería	Funciones usadas	Para qué
PyTorch (`torch.cuda`)	`torch.device("cuda")`, `.to(device, non_blocking=True)`, `torch.cuda.synchronize`, `max_memory_allocated`	Mover modelo y datos a la GPU, copias asíncronas y medición correcta
PyTorch DataLoader	`pin_memory=True`	Memoria fijada en RAM para copias CPU→GPU más rápidas
PyTorch AMP	`torch.autocast`, `torch.amp.GradScaler`	Precisión mixta en Tensor Cores sin perder estabilidad
cuDNN	`torch.backends.cudnn`	Kernels optimizados de convolución, BatchNorm y pooling
Numba CUDA	`@cuda.jit`, `cuda.grid`, `cuda.shared.array`, `cuda.syncthreads`, `cuda.to_device`, `cuda.event`	Kernel propio con grid y bloques explícitos, memoria compartida y medición con eventos
NVML / nvidia-smi	`pynvml`, `nvidia-smi`	Uso de GPU, VRAM y potencia como evidencia
CPU	`ProcessPoolExecutor`, `multiprocessing.shared_memory`, OpenCV, `numba.prange`	Preparación paralela de imágenes y versión multinúcleo del kernel

7. Conclusiones

- Separar el pipeline funcionó: la preparación en CPU con varios procesos fue 3.2× más rápida con la muestra y 4.6× con el dataset completo, y el entrenamiento de la CNN profunda en GPU fue 38× más rápido que en un hilo (59× en el modo optimizado). El pipeline completo pasó de 28 minutos a 46 segundos (37×).
- La paralelización en CPU tiene un techo cercano a los 8-12 procesos por el costo de crear procesos, el disco y la memoria compartidos y la mezcla de núcleos; agregar más procesos empeora el tiempo.
- La GPU acelera mucho más que los 24 hilos de CPU (12.5× a 19×) porque la red neuronal es cálculo regular y masivo, y el accuracy no cambia.
- El mayor enemigo de CUDA es mover datos: en el kernel aislado las copias fueron el 86 % del tiempo, y mantener los datos en VRAM fue la optimización más rentable del entrenamiento.

8. Distribución del trabajo

Integrante	Parte
Leonardo Mejía	Configuración, carga de datos y modelos del laboratorio, preparación secuencial, medición de tiempos y análisis de speedup
María José Girón	Preparación paralela con procesos y memoria compartida, kernel CUDA propio y análisis de operaciones y diseño
Cindy Gualim	Entrenamiento en GPU con CUDA, monitoreo de recursos, medición del kernel y evidencia de ejecución